

FLIGHT DELAY PREDICTION

Predicting Arrival Delay from Pre-Departure Schedule Data

Item	Detail
Course	Programming in Python
Assessment	Final-Term Project
Semester	Summer 25-26
Pathway	Classification
Instructor	Md.kamrul Islam

Group Members

#	Full name	Student ID	Ownership area
M1	ESRAT	23-50916-1	Data provenance, audit, documentation
M2	AUVROW	22-48920-3	EDA, split design, preprocessing, leakage control
M3	NABIL	24-60205-3	Baseline, models, tuning
M4	AREFIN	22-48766-3	Evaluation, plots, error analysis, reproducibility

1. Project Title and Group Information

Project title: FLIGHT_DELAY_PREDICTOR: Predicting Arrival Delay from Pre-Departure Schedule Data

Course: Programming in Python,

Final-Term Project, Summer 25-26

Instructor: Md.Kamrul Islam

Member details and contribution summary

A full contribution statement appears in Section 12. This table is the summary.

Member	Name	ID	Primary responsibility
M1	ESRAT	23-50916-1	Dataset sourcing, leakage-column removal, target construction, seven-check data audit
M2	AUVROW	22-48920-3	Exploratory analysis, time-aware split, manual preprocessing, six-check leakage suite
M3	NABIL	24-60205-3	Two baselines, three candidate models, cross-validated tuning, overfitting checks
M4	AREFIN	22-48766-3	Single final test evaluation, result figures, error and fairness analysis, reproducibility

2. Problem Statement, Stakeholders and Objective

2.1 The decision being made

The prediction exists to support one named actor making one named choice. We state it in those terms rather than as a modelling goal.

Decision: A passenger checking a booked flight the day before departure decides whether to arrange a later airport pickup, rebook a tight onward connection, or do nothing. The model supports that single decision by estimating whether the flight will arrive on time, moderately late (15 to 60 minutes), or severely late (over 60 minutes). Each answer maps to a different action, which is why we formulated the task with three classes rather than a single late / not-late flag.

2.2 The prediction moment

Our model predicts arrival delay BEFORE the aircraft departs. This is a deliberate design choice, not a limitation we failed to overcome, and it has one major consequence: every variable that only becomes known after push back is excluded from the feature set.

The most significant exclusion is the actual departure delay. In flight data, departure delay correlates with arrival delay at approximately 0.95, and a flight that leaves an hour late almost always arrives late. Including it would have produced a model with high accuracy that had learned essentially nothing, because the answer was already inside the question. Requirement FR-2 forbids this, and we treat it as the central integrity constraint of the project.

The full list of excluded variables, with the reason for each, appears in Section 5.2.

2.3 Unit of analysis

One scheduled commercial flight. Each row in the data set is one flight; each prediction concerns one flight.

2.4 Stakeholders

Different stakeholders want different things from the same model, and those wants conflict. Identifying the conflict is part of the analysis, because it determines which errors matter most.

Stakeholder	What they want	Which error hurts them
Passenger	Advance warning of disruption	A missed delay: they get no warning at all and miss a connection
Airline operations	Accurate resource planning	A false alarm: crew and gates reallocated unnecessarily
Airport scheduling	Congestion forecasting	Systematic error concentrated in peak hours

These interests are not aligned. A passenger would accept several false alarms to avoid one missed severe delay; an airline would not. We return to this trade-off in Section 9.2 when we rank our error types by severity.

2.5 What a useful result would mean

A useful result would mean a passenger receives a warning for a meaningful share of the flights that do in fact arrive severely late, at a false-alarm cost they would accept. Our results (Section 8.2) show 39.3% of major delays flagged against 0% for a model that does nothing, at the price of a warning on 39.7% of flights that in fact arrived on time.

That trade is only defensible for a stakeholder who values a missed warning above a false alarm. Section 9.2 states the ranking we adopted, and states plainly that an airline operations team would reasonably invert it. The model would not be useful to that stakeholder at this operating point, and we do not claim otherwise.

3. Dataset Source, Licence and Data Dictionary

3.1 Source and provenance

Item	Detail
Dataset name	Airline On-Time Performance Data, 2018 (distributed as 2018.csv)
Original publisher	US Bureau of Transportation Statistics (BTS), Office of Airline Information, United States Department of Transportation
Publisher URL	https://www.transtats.bts.gov/ (Reporting Carrier On-Time Performance, 1987-present)
Access route used	Obtained as a consolidated per-year CSV re-upload hosted on Kaggle, not downloaded field-by-field from the BTS portal. We cite BTS as the originating publisher and record the re-upload only as the access route, as requirement Section 4 directs.
Licence / terms	US Government work, published as a public record of mandatory carrier reporting. Exact licence line for the access :https://www.kaggle.com/datasets/yuanyuwendymu/airline-delay-and-cancellation-data-2009-2018
Rows in raw file	7,213,446 (full year 2018)
Date range covered	2018-01-01 to 2018-12-31 in the raw file; 2018-10-01 to 2018-12-31 in the subset actually analyzed
File size	After subsetting to October-December (1,796,121 rows); 600,002 rows analyzed after the documented row cap

3.2 Collection context

This is mandatory regulatory reporting, not a voluntary sample, and that distinction bounds what our conclusions can cover.

US carriers above a revenue threshold are legally required to report on-time performance to the Department of Transportation, so coverage of the seventeen reporting carriers present in our data is close to complete rather than self-selected. Nothing is missing because an airline chose not to submit it.

The threshold is also the limit of our reach. Carriers below it are absent from the file entirely, so no conclusion in this report extends to small regional or charter operators, and none extends to international departures, which this dataset does not cover.

3.3 Data dictionary summary

The complete data dictionary is submitted separately as data_dictionary.md. The features actually used by the model are:

Variable	Type	Unit	Meaning
FL_DATE	date	YYYY-MM-DD	Date of flight; drives the time-aware split
OP_CARRIER	text	2-letter code	Operating airline
ORIGIN	text	3-letter IATA	Departure airport
DEST	text	3-letter IATA	Arrival airport
CRS_ELAPSED_TIME	number	minutes	Scheduled journey length
DISTANCE	number	miles	Distance between airports
MONTH	integer	1-12	Derived from FL_DATE; captures seasonality
DAY_OF_WEEK	integer	0-6	Derived from FL_DATE; 0 = Monday
DEP_HOUR	integer	0-23	Derived from scheduled departure time
ARR_HOUR	integer	0-23	Derived from scheduled arrival time

3.4 Target variable definition

Class	Name	Condition	Justification
0	on_time	delay < 15 min	Official US DOT definition of a non-delayed flight
1	minor	15 to 60 min	Disruptive but most connections survive
2	major	over 60 min	Missed connections and rebooking become likely

On the 15-minute threshold. This is not our invention. It is the published US Department of Transportation standard, which makes it substantially easier to defend than a number we chose ourselves.

On the 60-minute threshold. This one is our judgement call, and we state that openly. Our reasoning is that beyond roughly an hour, missed connections and overnight rebooking become the dominant passenger consequence, which is a qualitatively different outcome from a delay that merely shortens a layover.

3.5 Observed class distribution

Class	Count	Percent of dataset
on_time	491,238	81.87%
minor	77,214	12.87%
major	31,550	5.26%

Imbalance ratio (largest class ÷ smallest class): 15.6 to 1 across the 600,002 rows analyzed (491,238 ÷ 31,550).

Within the training period alone the ratio is 16.5 to 1 (393,978 ÷ 23,898); within the test period it falls to 12.7 to 1 (97,260 ÷ 7,652). That difference between the two periods is itself a finding, and Section 9.5 returns to it as distribution shift.

4. Data Audit, Exploratory Analysis and Visualization Findings

4.1 Data quality audit

Missing values

Column	Missing count	Percent	Action taken	Why
Unnamed: 27	600,002	100.0%	Column dropped at load	An artefact of a trailing comma on every line of the source file. It carries no data at all. It reached the feature table once during development and crashed the run; the allow-list fix is described in Section 5.3.
ARR_DELAY (target source)	21,303 rows	3.43% of the subset	Rows removed	Without an outcome a row can be neither trained on nor tested against, so it is dropped rather than imputed. Imputing a target would be inventing the answer.
All six model features	0	0.0%	No action required	DISTANCE, CRS_ELAPSED_TIME, MONTH, DAY_OF_WEEK, DEP_HOUR and ARR_HOUR are complete across all 600,002 analyzed rows. The median imputation rule below therefore never fired on this data; it remains in the code because the rule must exist before it is needed, not after.

Our rule. Rows with a missing arrival delay are dropped, because without the outcome the row can be used neither for training nor for testing. Missing numeric features are filled with the training-set median. We

chose the median over the mean because the median is not dragged by extreme values: a single fourteen-hour flight would distort a mean noticeably and a median barely at all.

Duplicate rows

Duplicate rows found: 0

Percent of dataset: 0.00%

Duplicates are removed BEFORE the train/test split. The ordering is not incidental: if a duplicated flight were to land with one copy in training and the other in test, the model would be evaluated on a row it had already memorised, and the test score would be dishonest.

Impossible values

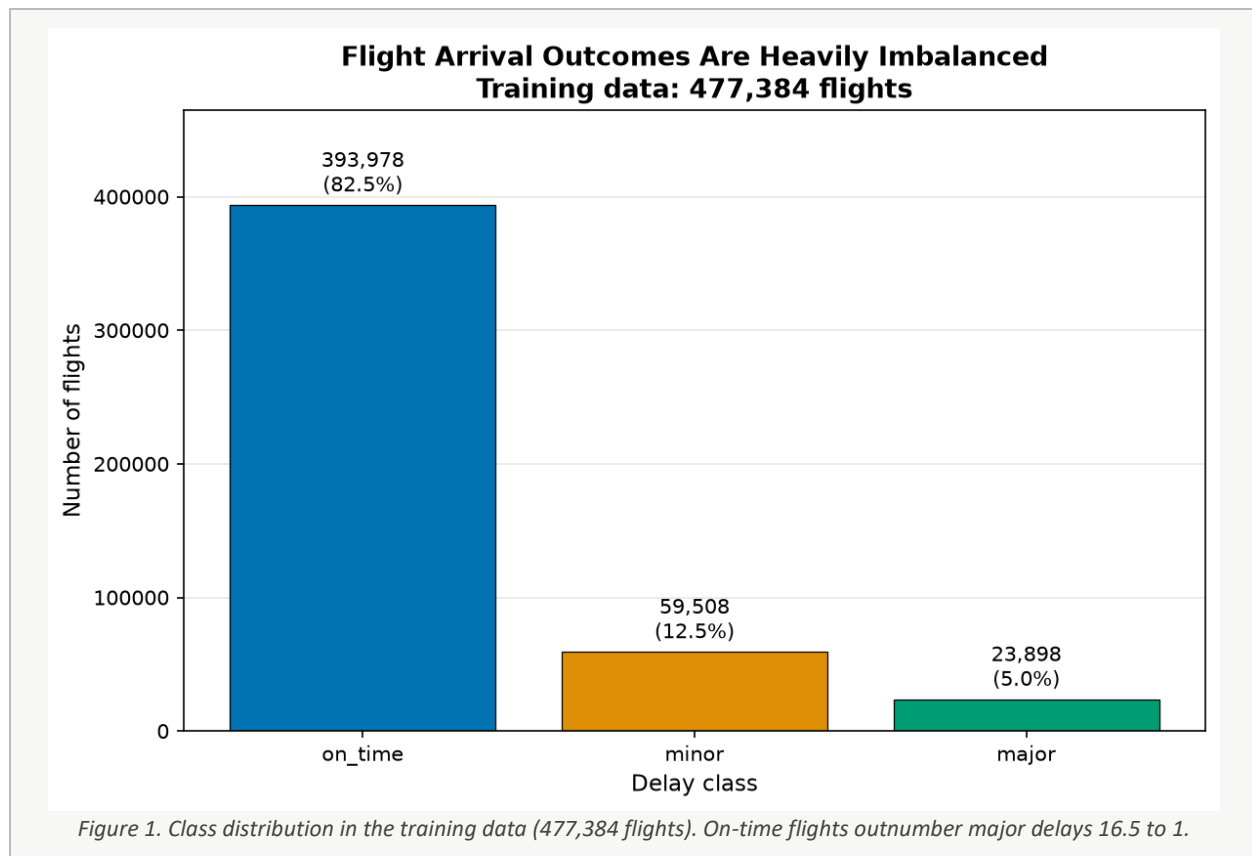
Column	Validity rule	Violations	Action
DISTANCE	must be ≥ 1	0	None required. Observed range 31 to 4,983 miles.
CRS_ELAPSED_TIME	must be ≥ 1	1	Reported, not silently deleted. One flight carries a scheduled elapsed time of -99 minutes, which is a recording error in the source. It is 1 row in 600,002 and cannot move any result. We report it rather than quietly removing it, because an undocumented edit is exactly what Section 4 forbids.
DEP_HOUR / ARR_HOUR	0 to 23	0	None required. Both span the full 0-23 range.
MONTH	1 to 12	0	None required. Values are 10, 11 and 12 only, as expected for the subset.

Outliers

Outliers were identified using the interquartile range rule and flagged for inspection, not deleted automatically. An unusually long flight is unusual but genuine. We remove a value only where we can articulate why it is wrong, as requirement Section 4 demands that no value be edited without a documented rule.

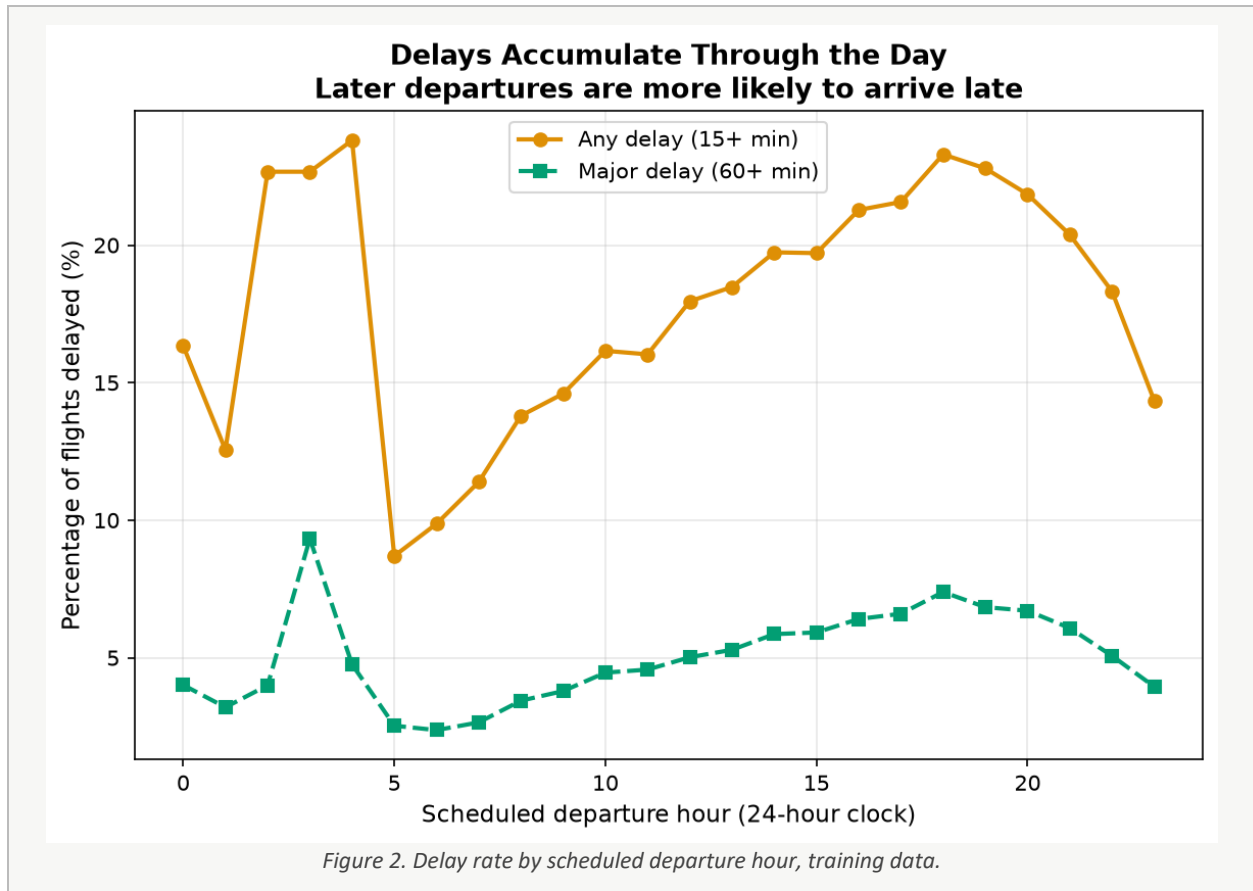
Column	Outliers found	Percent	Decision and reasoning
DISTANCE	35,782	5.96%	Kept. Every flagged value is a genuine long-haul domestic sector; the IQR fence sits at 2,028 miles and transcontinental routes exceed it legitimately. The rule flags unusual values, not wrong ones, and deleting real data to tidy a distribution would bias the model against exactly the routes where delay behaviour differs most.

Column	Outliers found	Percent	Decision and reasoning
CRS_ELAPSED_TIME	33,161	5.53%	Kept, for the same reason and largely the same flights. The one genuinely invalid value in this column (-99) is handled under impossible values above, not here.
MONTH, DAY_OF_WEEK, DEP_HOUR, ARR_HOUR	0	0.00%	None flagged. All four are bounded by construction, so the IQR fences fall outside their possible range.



What this shows and what we did about it: The distribution is severely imbalanced. On-time flights outnumber major delays by 16.5 to 1 in the training data, and the two delay classes together account for 17.5% of flights. The methodological consequence is immediate: a model predicting `on_time` for every flight scores 79.3% accuracy on our test set while detecting no delay whatsoever. We therefore fixed macro F1 as the tuning and selection metric before training anything, report class-wise precision and recall for every model in Section 8.4, and applied balanced class weighting to the two models that support it.

4.2 Exploratory analysis



Finding: The delay rate climbs monotonically through the working day, from 8.7% for 05:00 departures to 23.3% at 18:00, a 2.7-fold increase driven by nothing but the clock. The mechanism is aircraft rotation: one airframe flies several sectors a day, so a morning delay propagates into every subsequent leg, while the first departure of the day starts clean. This confirmed DEP_HOUR as genuinely informative rather than decorative, and the Random Forest subsequently ranked it the single most-used feature (Figure 10, importance 0.155). It also explains the error pattern in Section 9.3. The very early hours (02:00-04:00) show rates around 23% on fewer than 100 flights each, so we read those points as noise, not signal.

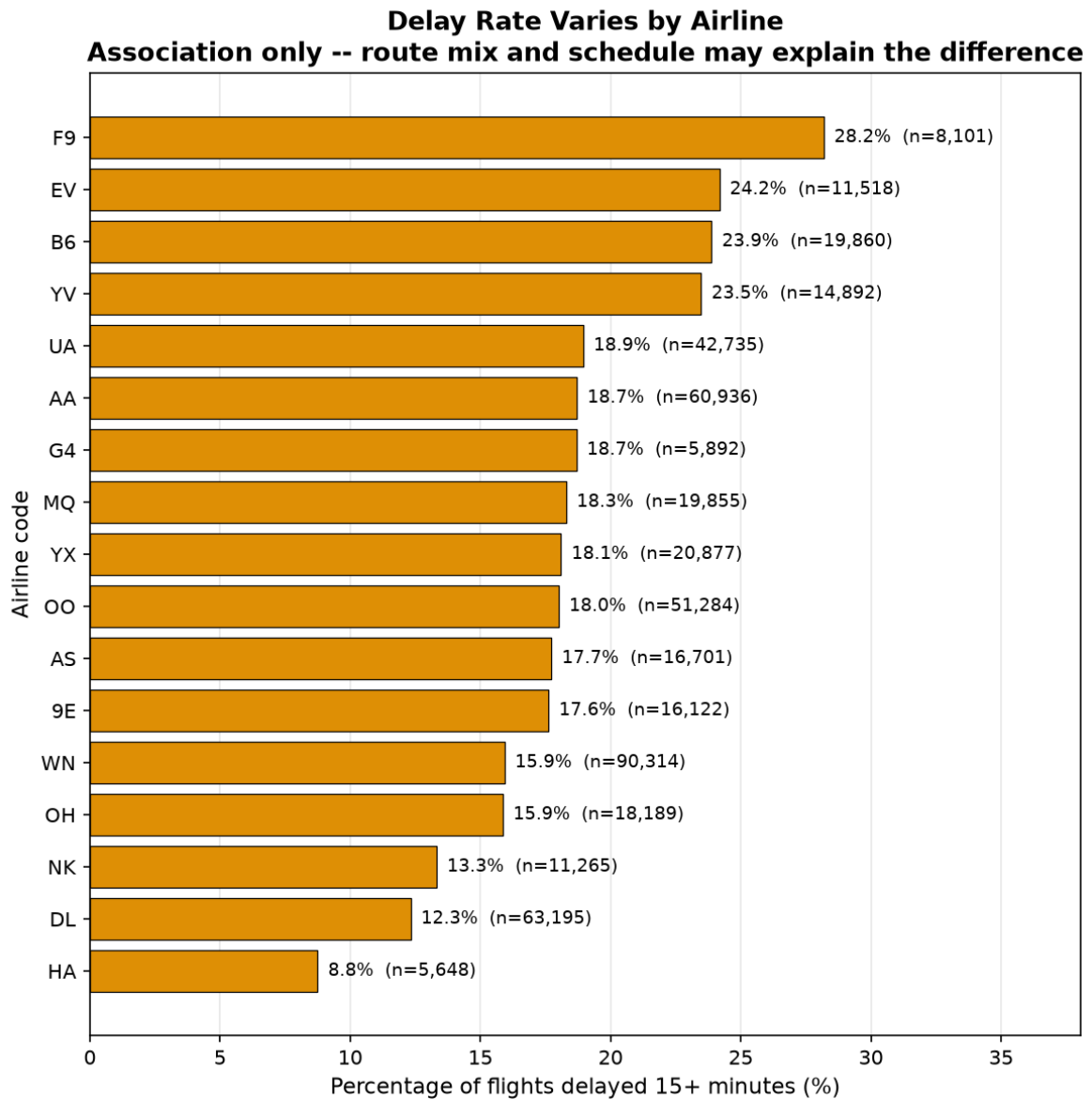
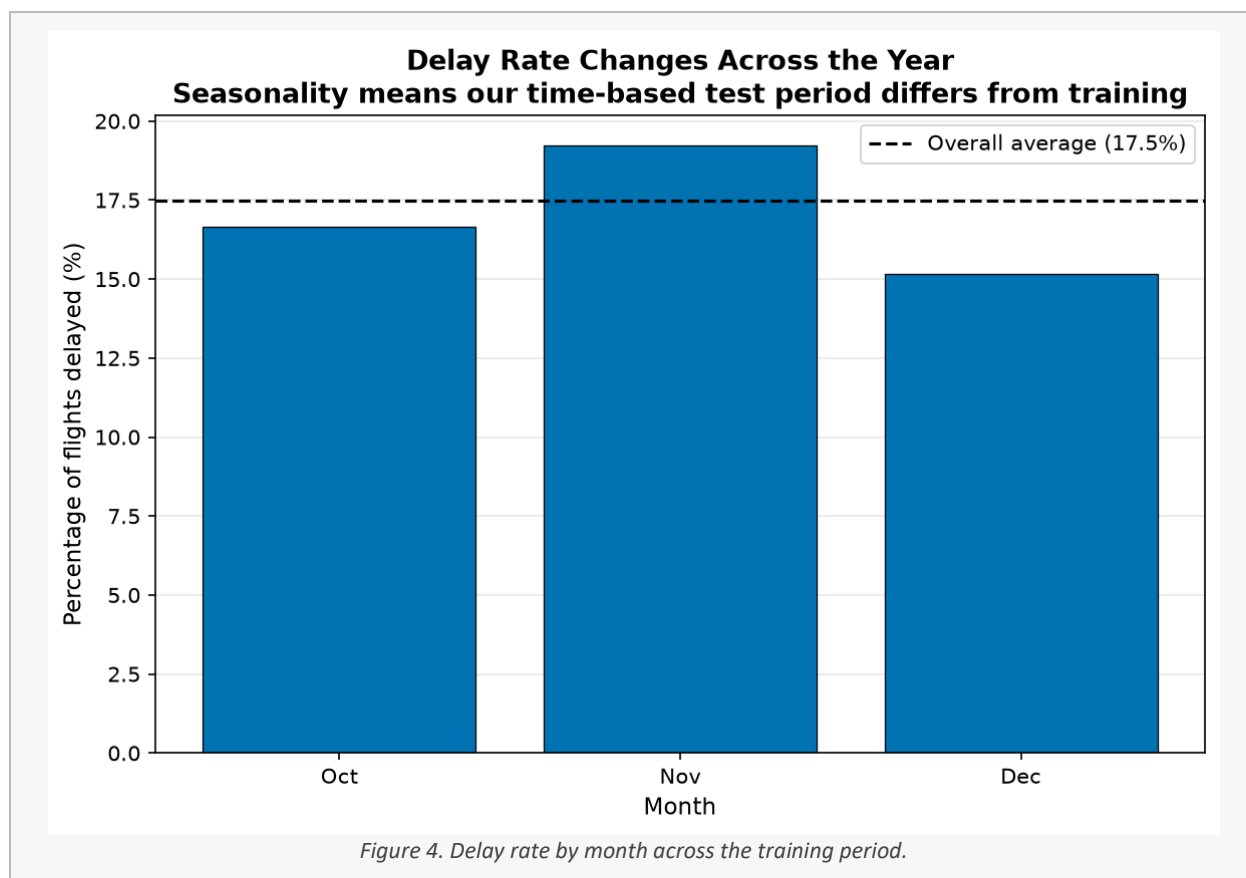


Figure 3. Delay rate by airline, training data. Association only; see the caution below.

Finding: Delay rates are associated with carrier across a wide range, from 8.75% (HA) to 28.19% (F9), a factor of 3.2. Major-delay rates spread similarly, from 1.10% to 9.67%. We use carrier as a feature on that basis, and the model does use it: OP_CARRIER_DL and OP_CARRIER_WN are the highest-ranked categorical features in Figure 10.



Finding: The training period is not seasonally flat. Delay rates run 16.6% in October, 19.2% in November and 15.1% across the part of December inside the training window, against a 17.5% training average. The test period, 13 to 31 December, has a true delay rate of 20.68%, higher than any month of the training data. The model is therefore evaluated on a harder period than it was trained on, quantified in Section 9.5.

This figure matters more than it first appears. Our train/test split is chronological, so the test period covers different months from the training period. If delays are strongly seasonal, the model is being evaluated under conditions it was not trained on. This is distribution shift, and we return to it in Section 9.5.

5. Split Strategy, Preprocessing Decisions and Leakage Controls

5.1 Why the split is time-aware, not random

Flight data is ordered in time, and we split it by date rather than at random. The earliest 80% of days form the training set; the latest 20% form the test set.

A random split would scatter flights across the calendar, placing some December flights in training and some January flights in testing. The model would then be learning from the future in order to predict the past, which is not how it could ever be deployed. In deployment you know the past and are guessing about tomorrow, and the evaluation must mirror that.

We split on whole days, not on rows. Splitting by row could divide a single day, placing some of its flights in training and the rest in testing. Flights sharing a day also share that day's weather and congestion, so a within-day split would leak information across the boundary. Splitting whole days removes that channel entirely.

Split parameter	Value
Method	Chronological, by whole calendar days
Test proportion	20% of days (19 of 92 days)
Cut-off date	Training ends 2018-12-12; test begins 2018-12-13
Training days / rows	73 days (2018-10-01 to 2018-12-12) / 477,384 rows
Test days / rows	19 days (2018-12-13 to 2018-12-31) / 122,618 rows

5.2 Excluded variables (leakage control, layer 1)

The following columns are deleted at load time, before any analysis takes place. Removing them from the table entirely means no later stage, and no team member, can use one by accident.

Column	What it is	Why excluded
DEP_DELAY	Actual departure delay	Correlates ~0.95 with arrival delay. The single most damaging leak available in this dataset.
DEP_TIME	Actual departure time	Known only after pushback
TAXI_OUT / TAXI_IN	Ground movement times	Post-departure and post-arrival
WHEELS_OFF / WHEELS_ON	Actual take-off and landing	Post-departure
ARR_TIME	Actual arrival time	This is the outcome itself
ACTUAL_ELAPSED_TIME	Real journey duration	Post-arrival
AIR_TIME	Real airborne duration	Post-arrival
CANCELLED / DIVERTED	Disruption flags	Outcome variables

Column	What it is	Why excluded
CARRIER_DELAY etc.	Official delay-cause breakdown	These fields are only non-zero if a delay occurred, so their presence encodes the answer

Also excluded, for a different reason

CRS_DEP_TIME and CRS_ARR_TIME are legitimately pre-departure, yet we still removed them. Both are stored as clock integers, where 1435 denotes 14:35. Treated as numbers these behave badly in two distinct ways. First, the arithmetic gaps are false: 13:59 and 14:00 are one minute apart in reality but 41 apart numerically. Second, their magnitude reaches roughly 2400 while our scaled features sit near zero with unit spread, so in a distance-based model such as K-Nearest Neighbours these two columns would overwhelm every other feature. We replaced them with DEP_HOUR and ARR_HOUR, which capture the useful signal and are scaled with everything else.

5.3 Preprocessing (leakage control, layer 2)

Step	What is learned (training only)	What is applied
Imputation	The median of each numeric column	Gaps filled in both sets with the training median
Scaling	The mean and standard deviation of each column	Both sets standardised using the training statistics
Encoding	Which category values are frequent enough to keep	Both sets one-hot encoded against the training vocabulary

The encoding step deserves particular attention. Deciding which airports are "busiest" by examining the full dataset would allow the test set to influence the shape of the feature space. It feels harmless and it is still leakage. Our vocabulary is built from training rows alone, and any category appearing only in the test set is routed to an OTHER bucket rather than causing an error.

5.4 Justifying the manual approach

Requirement DW-6 recommends using a pipeline "where practical". We wrote our preprocessing explicitly instead, and we accept that this requires justification.

Our reasoning was pedagogical and defensive. A Pipeline enforces the learn-from-training rule automatically, but it does so invisibly; a team member reading the code sees a configuration object rather than the fit and transform steps themselves. Requirement Section 18 requires every member to explain and defend preprocessing during the viva, and we judged that explicit code made that materially more achievable.

We recognise the cost of that choice. Writing the rule by hand means correctness rests on our discipline rather than on the library's guarantees. To cover this we implemented an automated verification suite,

described next. Our position is that a manual implementation with automated proof is at least as defensible as a pipeline taken on trust, and considerably more defensible in a viva.

5.5 Automated leakage verification (layer 3)

Rather than asserting that we were careful, we test it. The module `src/leakage_check.py` runs six checks and halts the entire workflow if any fails, because results produced by a leaking model are worthless and continuing would waste everyone's time.

#	Check	What it proves
1	No forbidden columns among features	No post-departure variable survived into the model, including as a one-hot derivative
2	Scaling learned from training only	Recomputes every mean from training rows alone and confirms it matches what was used
3	Category vocabulary from training only	Rebuilds the kept-category lists from training rows and confirms they match
4	Unseen test categories handled	Categories appearing only in test are bucketed to OTHER without error
5	No single flight appears in both training and test	Training and test share zero calendar dates, and the split is by whole day, so no individual flight can sit on both sides. An earlier version of this check compared feature vectors and flagged a 21.78% collision rate; we did not raise its threshold to make it pass, we replaced it with the assertion that actually matters and now report the collision rate separately as an INFO measurement (Figure 5, Section 9.7).
6	Time order holds	Every training date precedes every test date

We verified that these checks can actually fail. A test that never fails proves nothing, so we deliberately injected a forbidden column into the feature table and confirmed that check 1 detected it and stopped the run.

Figure 5. Result of the automated leakage verification suite, reproduced in full from results/metrics/leakage_checks.csv.

#	Check	Result	Detail
1	No forbidden (post-departure) columns among the features	PASS	Checked 84 features against 18 banned names. Found: none.
2	Scaling settings came from TRAINING data only	PASS	Every mean recomputed from training rows alone. All match.
3	Category vocabulary came from TRAINING data only	PASS	Each category list rebuilt from training rows alone. All match.
4	Unseen test categories safely bucketed into OTHER	PASS	OP_CARRIER 2, ORIGIN 315, DEST 316 unseen values, all routed to OTHER without error.
5	No single flight appears in both training and test	PASS	Training and test share 0 calendar dates; the split is by whole day.
6	Every training date comes before every test date	PASS	Latest training flight 2018-12-12, earliest test flight 2018-12-13.

--	Feature-collision rate between training and test	INFO	23,296 of 106,977 unique test feature vectors (21.78%) also occur in training. Reported as a measurement, not a pass or fail. Discussed in Section 9.7.
----	---	------	---

6. Baseline Definition and Result

6.1 What our baselines are

Baseline	Behaviour	What it represents
Majority class	Always predicts on_time, ignoring every feature	The score achievable with no learning at all
Stratified random	Guesses randomly in the training class proportions	Knowing the overall delay rate but nothing about the specific flight

6.2 Why a baseline is indispensable here

Because roughly 79% of flights arrive on time, a model that always answers "on_time" scores approximately 79% accuracy while detecting not one delay. Presented without context, that number looks like a competent result.

This is the reason requirement DW-7 makes a baseline compulsory. The majority baseline reached 79.32% accuracy on our test set with a macro F1 of 0.2949 and recall of exactly zero on both delay classes. It is, by construction, incapable of warning any passenger about any delay, and it still posts the highest accuracy figure in this report, higher than any model we actually trained. That single fact converts an impressive-sounding number into a visibly meaningless one, and it is the reference against which every later claim here is made.

6.3 Baseline results

Baseline	Accuracy	Macro F1	Recall on_time	Recall minor	Recall major
Majority class	0.7932	0.2949	1.0000	0.0000	0.0000
Stratified random	0.6757	0.3314	0.8257	0.1235	0.0470

7. Candidate Models, Reasoning, Hyperparameters and Tuning

7.1 Models selected

We compare three models, all drawn from the syllabus list in requirement Section 6, so no additional off-syllabus justification is owed. We chose these three deliberately because they fail in different ways, which makes the comparison informative; three similar models would have told us very little.

Model	How it works	Its characteristic weakness
Logistic Regression	Draws straight boundaries between classes	Cannot represent curved boundaries or interactions
K-Nearest Neighbours	Votes among the k most similar past flights	Degrades as feature count rises; no class weighting available
Random Forest	Averages many decision trees built on random data slices	Overfits if unconstrained; least interpretable

7.2 Model defence: Logistic Regression

Required point (Section 6)	Our answer
1. Why this task	Multi-class classification is precisely what it performs, via a one-vs-rest scheme across our three classes.
2. Why this data	Tabular data of moderate width. Its coefficients yield a readable statement about which scheduled factors associate with delay.
3. Preprocessing required	Scaled numeric features, without which coefficients are not comparable, and one-hot encoded categoricals.
4. Hyperparameter selection	C, the regularisation strength, selected by cross-validation on training data only.
5. Trade-offs	Most interpretable and fastest of the three. Restricted to linear boundaries.
6. Versus baseline	Yes, clearly, on the metric we committed to. Macro F1 0.3468 against 0.2949 for the majority baseline and 0.3314 for the stratified baseline, and recall of 0.4327 on the major class, the highest of any model here, against 0.0000 for the majority baseline. It loses to Random Forest on macro F1 (0.3468 vs 0.3752) and on cross-validated score (0.3324 vs 0.3427). It is not the selected model, but it is the one to prefer if catching severe delays outweighs everything else.

7.3 Model defence: K-Nearest Neighbours

Required point (Section 6)	Our answer
1. Why this task	Similarity-based classification with a defensible intuition: flights on comparable routes at comparable times plausibly behave alike.
2. Why this data	All features are numeric post-preprocessing, so distances are computable. However, one-hot encoding produced a wide feature space, which is a genuine problem here.

Required point (Section 6)	Our answer
3. Preprocessing required	Scaling is essential, not optional. Unscaled, DISTANCE with values in the thousands would dominate DEP_HOUR with values from 0 to 23, and the model would effectively ignore every other feature.
4. Hyperparameter selection	k, the number of voting neighbours. Small k overreacts to noise; large k blurs genuine distinctions. Selected by cross-validation on training data only.
5. Trade-offs	Slowest at prediction time. Suffers the curse of dimensionality across our feature count, where distances between points converge and 'nearest' loses meaning.
6. Versus baseline	No. KNN is the only model here that fails our stated criterion. Macro F1 0.3004 against the majority baseline's 0.2949, a margin of 0.0055 that is smaller than we would defend as real, with recall of 0.0043 on major and 0.0047 on minor. It answers on_time to 99.6% of test flights and has effectively collapsed into the baseline while costing 79.6 seconds to predict against the baseline's 0.001. It is worse than both other models on every count. This is exactly the outcome predicted below on structural grounds, and we report it as a result rather than dropping the model.

Documented model-specific difference

KNeighborsClassifier provides no class_weight parameter. The algorithm classifies by counting votes among nearest neighbours, and there is no point in that mechanism at which a class can be weighted. Our other two models use balanced class weighting; KNN structurally cannot.

We therefore predicted in advance that KNN would under-predict the rare classes, and we report this as a documented difference under Section 8 rather than as unfair treatment. Section 9 records whether that prediction held.

7.4 Model defence: Random Forest

Required point (Section 6)	Our answer
1. Why this task	Handles multi-class classification directly and copes with imbalance when class weighting is applied.
2. Why this data	Delay patterns are unlikely to be linear. The effect of departure hour plausibly differs by airport and by season, and trees capture such interactions without our having to specify them.
3. Preprocessing required	Trees split on thresholds rather than distances and so do not require scaling. We scaled regardless, so that all three models receive identical inputs, a condition Section 8 imposes for fair comparison.
4. Hyperparameter selection	max_depth and min_samples_leaf govern overfitting; n_estimators governs stability. Selected by cross-validation on training data only.
5. Trade-offs	Typically the strongest performer and the least interpretable. Its feature-importance scores are biased toward high-cardinality features and must be read with that caveat.

Required point (Section 6)	Our answer
6. Versus baseline	Yes, on both counts, and it is our selected model. Macro F1 0.3752 against 0.2949 for the majority baseline (+0.0803) and against 0.3468 for Logistic Regression (+0.0284). It leads on cross-validated score too (0.3427 vs 0.3324), and that CV gap of 0.0103 is roughly four times the fold-to-fold standard deviation of 0.0025, so the win is larger than our own measurement noise (Section 9.8). It predicts in 0.38 seconds against KNN's 79.6. Its cost is interpretability, addressed in Section 8.6.

7.5 Tuning method

All tuning was performed by cross-validation within the training data. The test set was not consulted at any point during model selection. Requirement Section 14 classifies repeatedly inspecting test performance and then changing the model as data leakage.

Setting	Value	Reason
Cross-validation folds	3	Balances reliability against runtime. With KNN tuning alone costing 1,226 seconds at 3 folds, 5 folds was not affordable inside the demo constraint of Section 18.
Fold strategy	Stratified	Preserves class proportions in each fold; essential given our 16.5-to-1 imbalance
Tuning metric	f1_macro	Averages F1 across all three classes equally, so ignoring a rare class is penalised heavily. Fixed before training, not chosen after seeing which model it favoured.
Random seed	42	Fixed across splitting, sampling and every stochastic model (config.RANDOM_STATE)

7.6 Full tuning log

Figure 6. Every hyperparameter configuration evaluated, including those that performed poorly. Reproduced in full from results/metrics/tuning_log.csv.

Model	Configuration	Mean CV f1_macro	SD across folds	Mean train score	Fit time (s)	Outcome
logistic_regression	C = 0.1	0.3324	0.0012	0.3335	18.53	selected
logistic_regression	C = 1.0	0.3324	0.0010	0.3335	18.64	--
knn	n_neighbors = 15	0.3134	0.0007	0.3254	2.06	selected
knn	n_neighbors = 35	0.3046	0.0008	0.3060	2.16	--
random_forest	max_depth = 10, n_estimators = 100	0.3176	0.0011	0.3244	56.13	--
random_forest	max_depth = 20, n_estimators = 100	0.3427	0.0025	0.3697	87.55	selected

The two losing configurations are retained deliberately. The fit-time column records per-configuration model fitting only.

8. Final Metrics, Diagnostic Plots, Model Comparison and Computational Cost

8.1 Evaluation protocol

The test set was evaluated exactly once, after every modelling decision had been finalised. No result in this section was used to revise a model, a parameter or a feature. Requirement Section 14 requires the test set to remain untouched until final evaluation, and we treated the reported figures as final the moment they were produced.

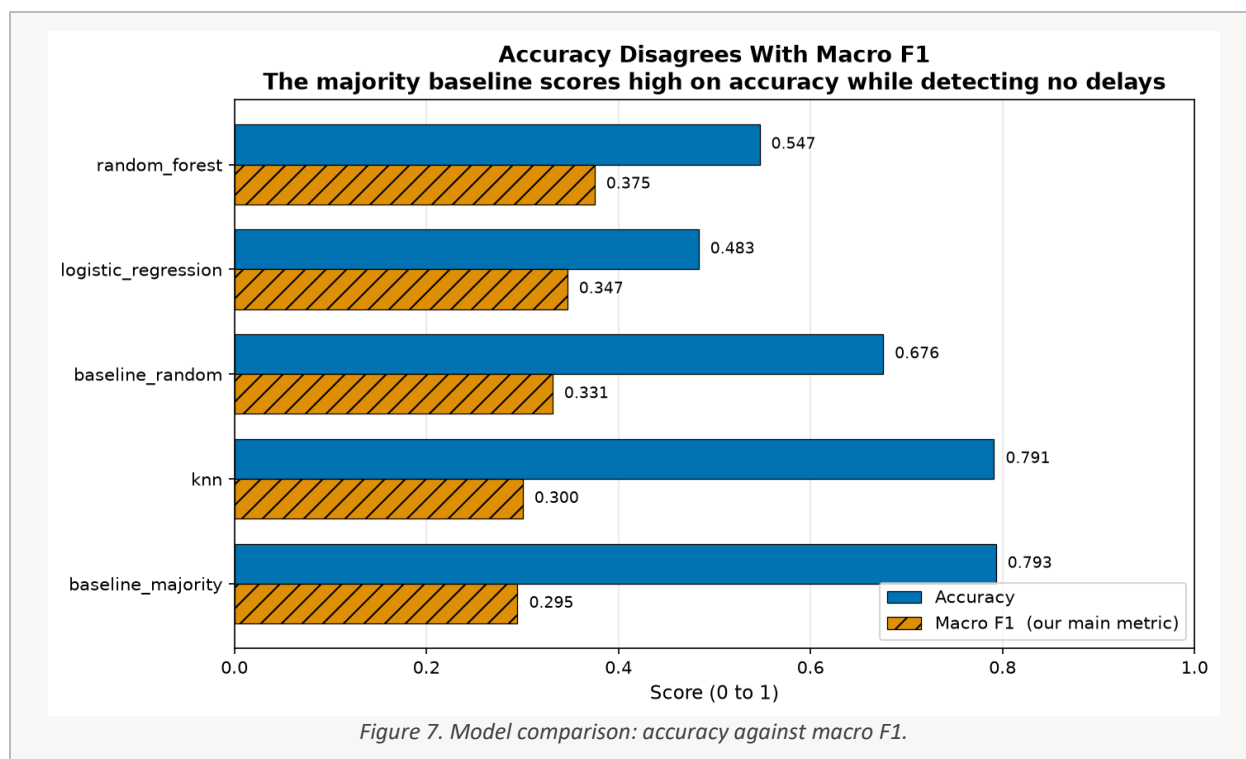
Test set size (flights): 122,618 (97,260 on_time, 17,706 minor, 7,652 major)

Test period covered: 2018-12-13 to 2018-12-31, 19 consecutive days

8.2 Model comparison

Model	Accuracy	Macro F1	Recall on_time	Recall minor	Recall major
Majority baseline	0.7932	0.2949	1.0000	0.0000	0.0000
Random baseline	0.6757	0.3314	0.8257	0.1235	0.0470
Logistic Regression	0.4835	0.3468	0.5178	0.3169	0.4327
K-Nearest Neighbours	0.7912	0.3004	0.9963	0.0047	0.0043
Random Forest (selected)	0.5470	0.3752	0.6026	0.3081	0.3928

All models were evaluated on the same split, with the same feature set, the same preprocessing logic and the same metric definitions, as required by Section 8 for a fair comparison. The single documented exception is class weighting, which K-Nearest Neighbours does not support (Section 7.3).



8.3 The accuracy trap, demonstrated

Majority baseline accuracy: 0.7932 (79.32%)

Best real model accuracy: 0.5470 (54.70%) for the selected Random Forest, 24.6 percentage points below the baseline

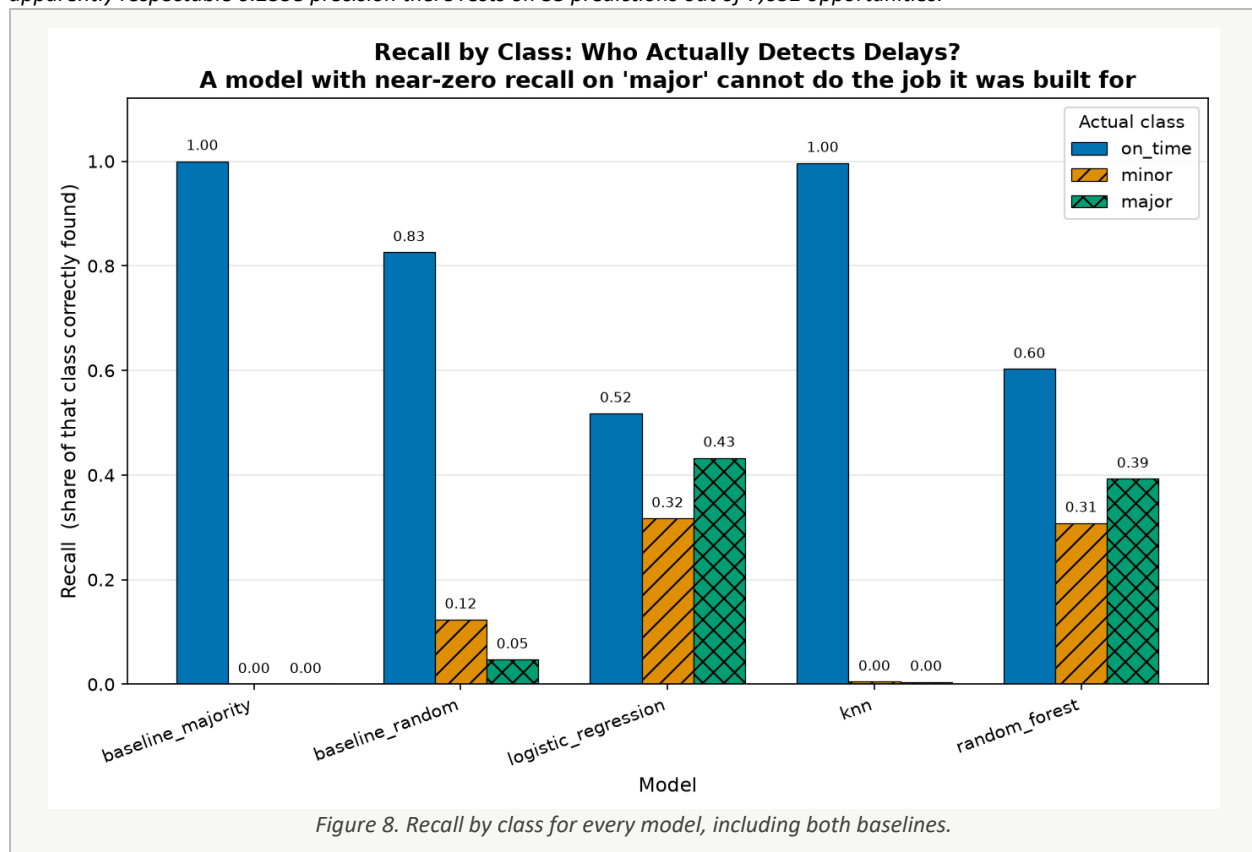
Majority baseline recall on 'major': 0.0000. It identifies none of the 7,652 severely delayed flights in the test set. Judged on accuracy alone, the correct conclusion would have been to deploy a model that does nothing at all. This is not a hypothetical hazard; it is what our own results table would have told us had we read it wrongly, and it is the clearest single justification for the metric choices set out in Section 7.5.

8.4 Per-class performance

Model	Class	Precision	Recall	F1	Test flights
baseline_majority	on_time	0.7932	1.0000	0.8847	97,260
baseline_majority	minor	0.0000	0.0000	0.0000	17,706
baseline_majority	major	0.0000	0.0000	0.0000	7,652
baseline_random	on_time	0.7933	0.8257	0.8092	97,260
baseline_random	minor	0.1438	0.1235	0.1329	17,706
baseline_random	major	0.0584	0.0470	0.0521	7,652
logistic_regression	on_time	0.8478	0.5178	0.6429	97,260
logistic_regression	minor	0.1888	0.3169	0.2367	17,706

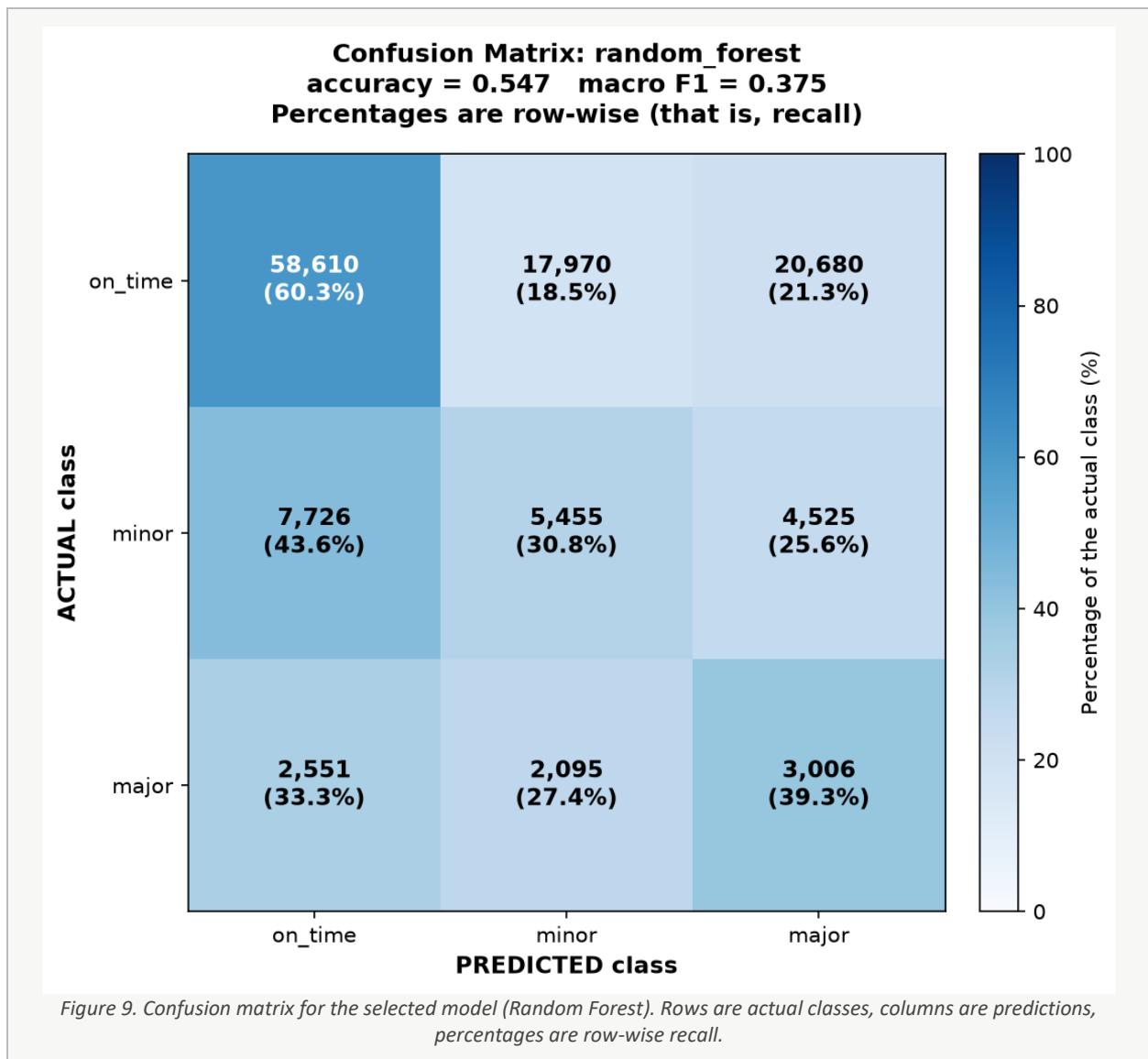
logistic_regression	major	0.0988	0.4327	0.1609	7,652
knn	on_time	0.7938	0.9963	0.8836	97,260
knn	minor	0.1995	0.0047	0.0092	17,706
knn	major	0.2558	0.0043	0.0085	7,652
random_forest (selected)	on_time	0.8508	0.6026	0.7055	97,260
random_forest (selected)	minor	0.2138	0.3081	0.2524	17,706
random_forest (selected)	major	0.1066	0.3928	0.1676	7,652

Table. Class-wise precision, recall and F1 for every model, from results/metrics/per_class_metrics.csv. Note the precision column on the major class: even the selected model is right only 10.66% of the times it issues a major-delay warning, and KNN's apparently respectable 0.2558 precision there rests on 33 predictions out of 7,652 opportunities.



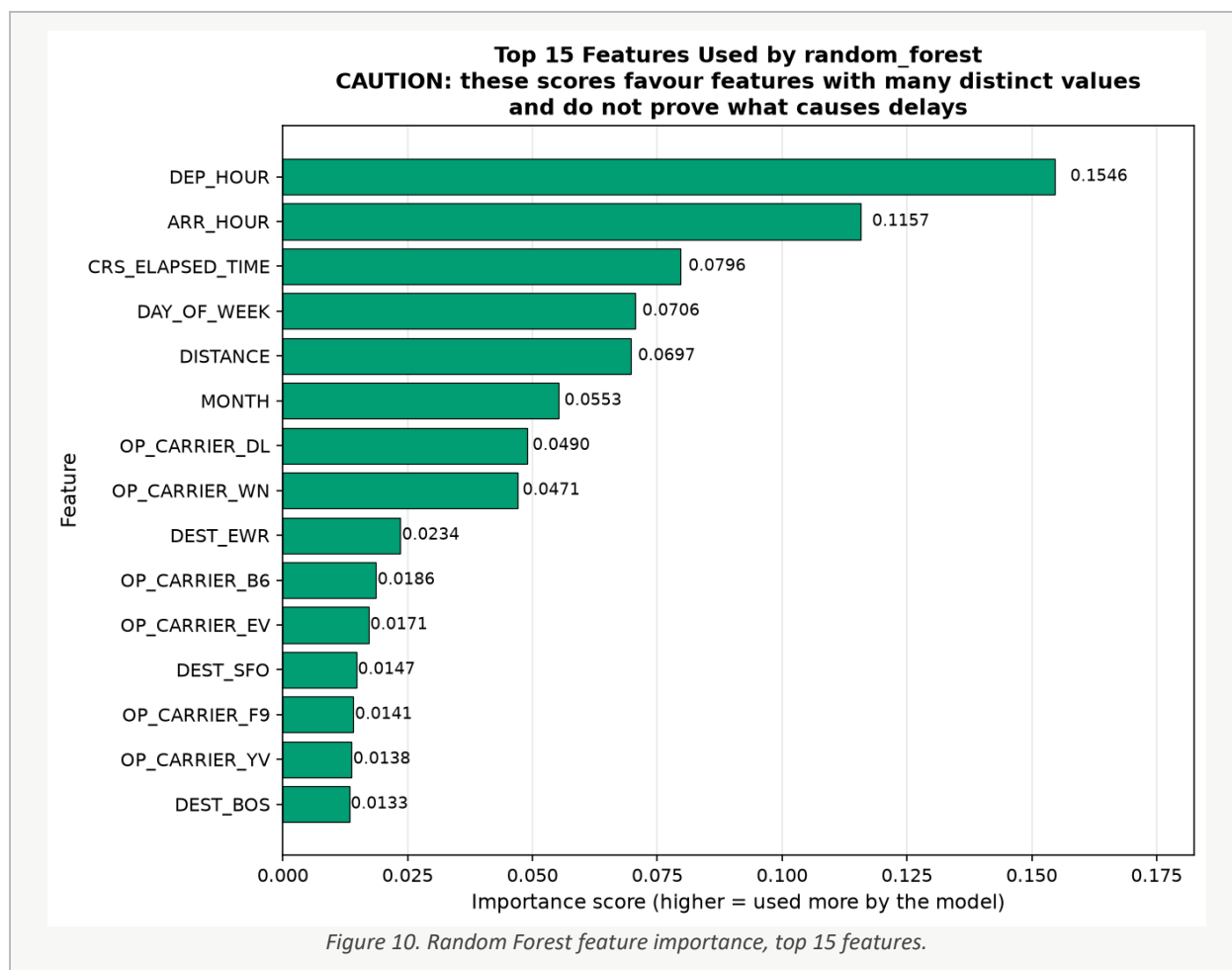
What this figure reveals: It separates models that are useful from models that merely score well. K-Nearest Neighbours posts 79.12% accuracy (second highest in the table) while registering recall of 0.0047 on minor and 0.0043 on major. It has collapsed into the majority baseline: 99.63% of its predictions on true on-time flights are on_time, and it finds 33 of 7,652 major delays. This is precisely the behaviour predicted in Section 7.3 on structural grounds, before the test set was opened, because KNN cannot apply class weighting. Logistic Regression and Random Forest sit in the opposite regime, trading on-time recall (0.5178 and 0.6026) for delay recall an order of magnitude above the baselines.

8.5 Confusion matrix for the selected model



Reading of the matrix: Of 7,652 genuinely major delays, the model flags 3,006 as major (39.3%), downgrades 2,095 to minor (27.4%) and misses 2,551 entirely as on_time (33.3%). Of 17,706 minor delays it recovers 5,455 (30.8%), while 7,726 (43.6%) are called on_time. The dominant error is in the top row: 38,650 of 97,260 on-time flights (39.7%) are given some delay warning. That is the price paid for the delay recall in the other two rows, and it is a deliberate consequence of balanced class weighting rather than an accident. Reading down the major column, 3,006 of 28,211 major predictions are correct (precision 0.1066), so a major warning from this model is right roughly one time in nine.

8.6 Feature importance



These scores must be read with a specific caveat. Random Forest importance is systematically biased toward features with many distinct values. Our one-hot columns take two values each, whereas DEP_HOUR and ARR_HOUR take 24, DISTANCE 1,473 and CRS_ELAPSED_TIME 474, and the ranking reflects that: the six numeric features occupy the top six places (DEP_HOUR 0.155, ARR_HOUR 0.116, CRS_ELAPSED_TIME 0.080, DAY_OF_WEEK 0.071, DISTANCE 0.070, MONTH 0.055) before any carrier or airport column appears (OP_CARRIER_DL 0.049). Part of that ordering is a structural artefact of the cardinality bias and not evidence about delay. We therefore treat this figure as an indication of what the model used, not as a ranking of what causes delay. The one reading we do draw is that it is consistent with Figure 2: the two hour-of-day features together account for 0.27 of total importance, and the EDA showed a 2.7-fold delay gradient across the day before any model was fitted.

8.7 Computational cost

Model	Training / tuning time (s)	Prediction time (s)	Notes
Majority baseline	< 0.1	0.0011	No learning of any kind

Model	Training / tuning time (s)	Prediction time (s)	Notes
Stratified baseline	< 0.1	0.0068	Samples from the training class proportions
Logistic Regression	27.0 (tuning stage, 2 configurations x 3 folds)	0.0777	Cheapest real model at both ends
K-Nearest Neighbours	1,226.4 (tuning stage, 2 configurations x 3 folds)	79.5876	84% of the entire workflow runtime. Fitting is trivial; the cost is at prediction, comparing each test flight against 477,384 training flights across 84 features. Roughly 200 times the prediction cost of Random Forest, for the worst macro F1 of any real model.
Random Forest (selected)	117.0 (tuning stage, 2 configurations x 3 folds)	0.3754	Best macro F1 and second cheapest prediction. No trade-off was required here.

Hardware used: Windows 11 (AMD64), Intel64 Family 6 Model 165, 12 logical cores, Python 3.14.6

Total workflow runtime: 24 min 25 s for the complete ten-stage run that produced every figure and table in this report

Row cap applied (MAX_ROWS): 600,000, giving 600,002 rows retained. Rows are selected by SAMPLING_METHOD = "stratified_by_day", which keeps the same share (about one flight in three) of every one of the 92 calendar days, so no day is lost and the monthly, weekday and time-aware structures all survive. Keeping the oldest 600,000 rows instead would have stopped in early November and discarded December, the month the model is tested on. This is a documented compute decision, not undisclosed filtering.

One measurement caveat. The tuning times above are whole-stage wall-clock figures, which include cross-validated prediction as well as fitting. The fit-time column inside the tuning log (Figure 6) records per-

configuration model fitting only, so the two columns are not directly comparable and should not be expected to agree.

8.8 Selected model

Final selected model: Random Forest (`n_estimators = 100`, `max_depth = 20`, `class_weight = "balanced"`, `random_state = 42`)

Selection justification: It leads on the metric we committed to in advance, macro F1 0.3752 against 0.3468 for Logistic Regression and 0.3004 for KNN, and it leads on the cross-validated score used to make the choice (0.3427 vs 0.3324 and 0.3134), a decision taken before the test set was opened. That CV margin of 0.0103 over Logistic Regression is about four times the fold-to-fold standard deviation of 0.0025, so it exceeds our own measurement noise (Section 9.8). Runtime does not argue against it: at 0.38 seconds to predict it is second cheapest of the three, and roughly 200 times faster than KNN, which finished last on quality. The one genuine cost is interpretability, and the honest counter-argument is that Logistic Regression catches more major delays (recall 0.4327 vs 0.3928): if the only thing that mattered were severe-delay recall, that model would win. We selected on the success criteria fixed in Section 2 rather than on the criterion our results happened to favour.

9. Error Analysis, Bias, Assumptions, Uncertainty and Limitations

9.1 Overall error profile

Test flights: 122,618

Correctly classified: 67,071 (54.70%)

Misclassified: 55,547 (45.30%)

9.2 Which mistakes matter most

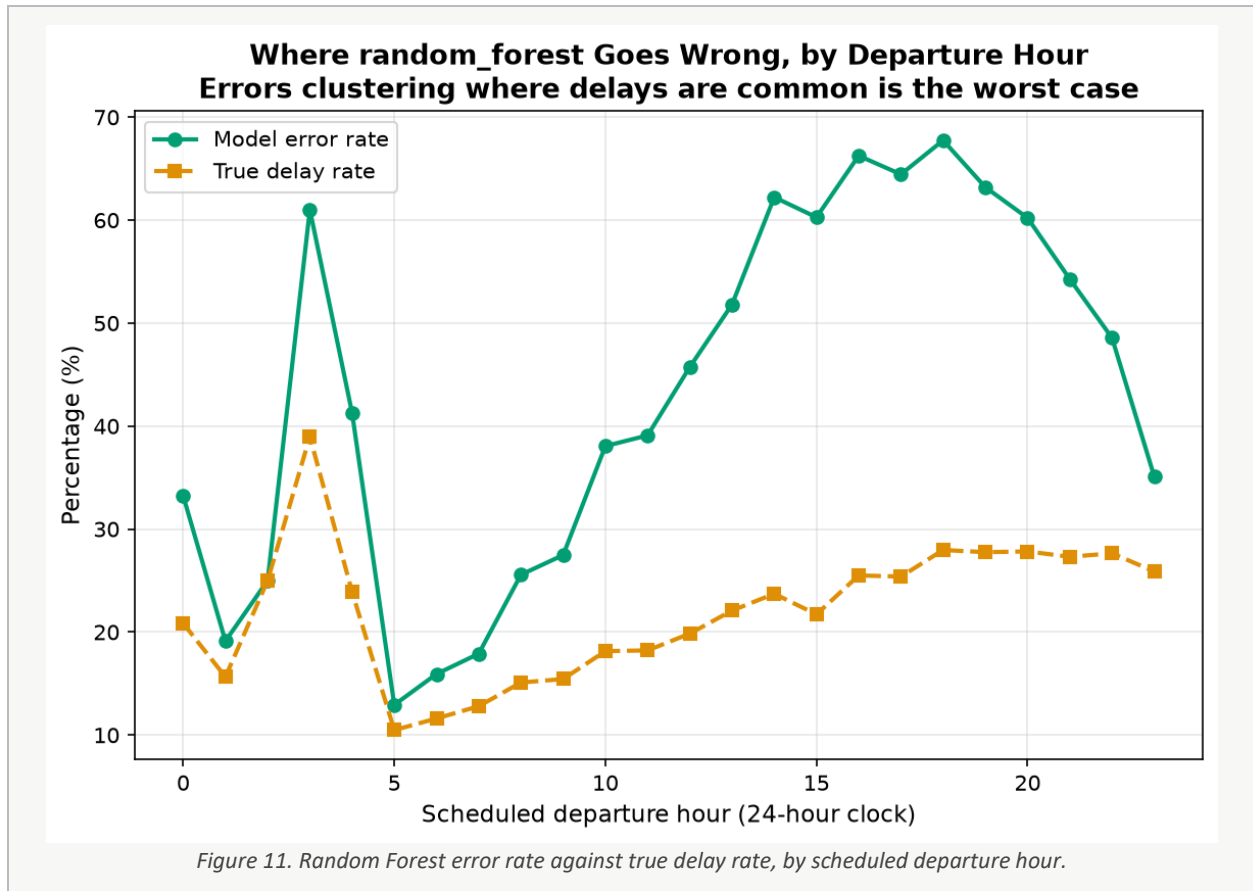
Not all errors carry equal cost, and averaging them into a single score conceals that. We classify our errors by consequence to the passenger stakeholder identified in Section 2.4.

Error type	What happened	Severity	Count
Missed major delay	Predicted on_time; flight was over an hour late	Most severe: no warning given at all	2,551 (4.59% of all errors)
Missed minor delay	Predicted on_time; flight was 15 to 60 minutes late	Serious: no warning given	7,726 (13.91%)
Wrong severity	Predicted a delay, but the wrong band	Tolerable: passenger was warned	6,620 (11.92%): 4,525 minor called major, 2,095 major called minor
False alarm	Predicted a delay; flight arrived on time	Tolerable: unnecessary concern caused	38,650 (69.58%): 20,680 called major, 17,970 called minor

This ranking is a judgement, not a mathematical fact. We state that openly. Our reasoning is that a passenger given no warning cannot act at all, whereas a passenger given an imprecise warning has still been prompted to check. An airline operations stakeholder would very likely rank false alarms as more costly than we have, and a different ranking would justify a different decision threshold.

Percentage of all errors that are missed major delays: 4.59% (2,551 of 55,547). Read the other way, the model gives no warning at all to 33.3% of the passengers on severely delayed flights. Nearly seven in ten of its mistakes are false alarms, which is the failure mode we chose to accept.

9.3 Where the model fails



Pattern observed: The two lines rise together, and the model's line rises faster. Error rate runs near 13% for 05:00 departures, where the true delay rate is 10%, and reaches 67.5% at 18:00, where the true delay rate is 28%. The model is least reliable at precisely the hours when delays are most frequent and a warning is worth most. The gap between the lines also widens with the hour: at 05:00 the model errs on roughly one flight in eight, by early evening on two in three. This is the least convenient possible failure mode, and no aggregate metric exposes it; only the hourly breakdown does. The spike at 02:00-04:00 sits on fewer than 100 flights per hour and we do not read it as a pattern.

Error rate by airline

Airline	Test flights	Error rate (%)	True delay rate (%)
EV	2,812	83.85	25.71
F9	2,012	83.20	27.29
YV	3,686	81.47	23.36
B6	5,394	65.20	28.20

Airline	Test flights	Error rate (%)	True delay rate (%)
OO	13,167	60.22	20.76
WN	23,842	36.74	24.97
HA	1,511	17.21	12.91
DL	15,282	13.57	11.95

Error rate by origin airport

Airport	Test flights	Error rate (%)	True delay rate (%)
[paste from ERROR RATE BY ORIGIN AIRPORT block of python -m src.error_analysis]			

9.4 Bias and fairness

The subgroup breakdowns above are not merely diagnostic. Where the model is measurably less accurate for particular carriers or airports, passengers flying those routes receive a worse service from it than others do.

We supplied no demographic variable of any kind. Nevertheless, airport codes encode geography, and geography correlates with income, population density and infrastructure investment. Systematically worse performance at smaller regional airports would therefore constitute a fairness problem arising entirely through proxy variables, with no protected attribute ever entering the model.

Disparity observed in our results: Large and systematic. The model is wrong for 83.85% of EV flights and 13.57% of DL flights, a factor of 6.2 between two groups of passengers using the same tool. The spread is not explained by how often those carriers are actually late: EV's true delay rate (25.71%) is close to WN's (24.97%), yet WN's error rate is 36.74% against EV's 83.85%. What separates them is volume in the training data. WN contributes 90,314 training flights and DL 63,195, while EV contributes 11,518 and F9 8,101, and the three worst-served carriers are all small regional or low-cost operators. A passenger on a regional carrier therefore receives a materially worse service from this model than a passenger on a major one, and the same mechanism applies to airports, where 318 of 348 codes are collapsed into a single OTHER bucket covering about 35% of rows. We report this as a fairness finding rather than a diagnostic footnote, and it is a reason not to deploy the model as it stands.

9.5 Distribution shift

Our test set covers a later time period than our training set, and Section 4.2 established that delay rates vary seasonally. The model is therefore evaluated under conditions that differ systematically from those it was trained on.

This is an honest consequence of time-aware splitting rather than a flaw in it. A random split would have concealed the problem while producing a more flattering and less trustworthy score. We regard the more pessimistic figure as the more informative one.

Months covered by training: October (full), November (full), December 1-12. Delay rates 16.6%, 19.2% and 15.1% respectively, against a 17.5% training average.

Months covered by test: December 13-31 only, the Christmas and New Year peak. True delay rate 20.68%, higher than any month in the training window.

Observed effect on results: The class balance itself shifts. On-time flights are 82.5% of training rows but 79.3% of test rows, and the imbalance ratio falls from 16.5-to-1 to 12.7-to-1. The model is fitted on a calmer period and scored on a busier one, which is a real and unfavourable difference, not a rounding effect. Two consequences follow. First, our reported figures are pessimistic relative to a model evaluated on an average period; we prefer that to a flattering number. Second, the by-month error breakdown is empty as an analysis: because the test window sits entirely inside December, `results/metrics/errors_by_month_random_forest.csv` contains exactly one row and supports no across-month comparison. We state that rather than presenting a one-row table as if it were a finding.

9.6 Assumptions

- That historical patterns continue to hold in the test period and beyond.
- That scheduled times in the source data are accurate as published.
- That the reporting carriers present in the dataset represent the flights we wish to reason about.

9.7 Limitations

Limitation	Consequence
No weather data	Weather causes a substantial share of real delays and is entirely absent from our features. This is the single largest gap in the feature set and likely the dominant ceiling on achievable performance.
Pre-departure framing	We deliberately excluded the strongest available predictor. Performance is therefore lower than a post-departure model would report, by design.
Row cap applied	A documented computational decision, recorded in <code>config.py</code> as <code>MAX_ROWS = 600,000</code> with stratified-by-day sampling. Not hidden filtering.
No aircraft rotation history	Delay propagation between consecutive flights of the same airframe is a known mechanism we could not capture.
Feature importance bias	Random Forest importances favour high-cardinality features and cannot be read as a causal ranking.
KNN cannot weight classes	A structural limitation of the algorithm, documented in Section 7.3.
Coarse features place a measured ceiling on accuracy	21.78% of unique test feature vectors also occur in training, and about 10% of repeated vectors carry more than one outcome. Where identical inputs genuinely map to different answers, no model can get both right. This caps the achievable score before any modelling decision is made. It is not leakage: training and test share zero calendar dates (Figure 5, check 5).

9.8 Uncertainty

Gap between best and second-best model (macro F1): 0.0103 on the cross-validated training score that actually decided the selection (Random Forest 0.3427, Logistic Regression 0.3324). On the held-out test set the gap is larger, 0.0284 (0.3752 vs 0.3468), but that figure played no part in the choice and is reported for completeness only.

Standard deviation across CV folds: 0.0025 for the selected Random Forest configuration, the largest of any configuration in the tuning log (Logistic Regression 0.0012, KNN 0.0007).

Is the gap larger than the fold-to-fold variation? Yes, by roughly a factor of four (0.0103 against 0.0025), so we do claim a real difference between Random Forest and Logistic Regression rather than a lucky draw.

We are deliberately more cautious one row down: KNN's macro F1 of 0.3004 exceeds the majority baseline's 0.2949 by 0.0055, and we do not present that as a meaningful improvement. Had the Random Forest margin been of that size, the honest conclusion would have been that the two models are tied, and we would have selected on interpretability and runtime instead, which Section 8 permits.

10. Reproducibility Information

10.1 Randomness control

Where randomness occurs	How it is controlled
Train/test split	Deterministic: split by date, no randomness involved
Cross-validation folds	StratifiedKFold with a fixed random_state
Random Forest	random_state fixed
Dummy baselines	random_state fixed
Row sampling (if random mode used)	random_state fixed

Global seed value (config.RANDOM_STATE): 42

10.2 Environment

Component	Version
Python	3.14.6
pandas	3.0.5
numpy	2.5.1
scikit-learn	1.9.0
matplotlib	3.11.1

Submitted as requirements.txt. Regenerate with: `pip freeze > requirements.txt`

10.3 Run instructions

From the project root directory:

Step	Command
1. Install dependencies	<code>pip install -r requirements.txt</code>
2. Place dataset	Copy the CSV into data/raw/
3. Configure	In config.py set REAL_DATA_FILENAME and USE_SAMPLE_DATA = False
4. Run everything	<code>python run_all.py</code>
5. Verify reproducibility	<code>python verify_reproducibility.py</code>

No file depends on an undocumented local path. Every path is constructed from the project root at runtime, so the workflow runs unchanged on any machine and from any directory.

10.4 Reproducibility verification

Asserting that a seed has been set is a claim; running the workflow twice and showing identical outputs is evidence. The script `verify_reproducibility.py` executes the full pipeline twice and compares every score and every tuning result.

Prediction timings are deliberately excluded from that comparison. Timings are measured with a stopwatch and vary with hardware and system load, so they can never match exactly.

10.5 File structure

Path	Purpose	Owner
<code>config.py</code>	All settings: paths, seeds, features, thresholds	M4
<code>run_all.py</code>	Executes the complete workflow in order	M4
<code>verify_reproducibility.py</code>	Proves two runs produce identical results	M4
<code>src/data_loading.py</code>	Load data, remove leakage columns, build target	M1
<code>src/data_audit.py</code>	Data quality checks	M1
<code>src/split.py</code>	Time-aware train/test split	M2
<code>src/eda.py</code>	Exploratory analysis and figures	M2
<code>src/preprocessing.py</code>	Imputation, scaling, encoding	M2
<code>src/leakage_check.py</code>	Automated leakage verification	M2
<code>src/models.py</code>	Model definitions and written defences	M3
<code>src/train.py</code>	Tuning and training	M3
<code>src/evaluate.py</code>	Final test-set evaluation	M4
<code>src/plots.py</code>	Result figures	M4
<code>src/error_analysis.py</code>	Error pattern analysis	M4

11. Conclusion, Practical Interpretation, Responsible Use and Future Work

11.1 What we found

Our selected model, a Random Forest, achieved a macro F1 of 0.3752 on a test set of 122,618 flights it had never seen, against 0.2949 for a majority-class baseline and 0.3314 for a stratified random one. It identifies 39.3% of severely delayed flights and 30.8% of moderately delayed ones, where the majority baseline identifies none of either. Its accuracy, 54.70%, is 24.6 percentage points below that same do-nothing baseline.

This is a modest result and we present it as nothing more. The explanation is structural rather than a defect in the implementation. The strongest available predictor of a late arrival is the departure delay, and we deleted it deliberately, along with sixteen other post-departure columns, because at our stated prediction moment the aircraft has not moved and none of those values exists. A model using them would have scored far higher and been useless. What remains is schedule information alone, which carries real but limited signal about an outcome driven substantially by weather and by cascading operational disruption, neither of which is in our data.

A second ceiling is measured rather than assumed. Because our features are deliberately coarse (times rounded to the hour, three months, 30 named airports plus OTHER), 21.78% of unique test feature vectors also occur in training, and about 10% of repeated vectors carry more than one outcome. Where identical inputs genuinely map to different answers, no model can get both right. That caps the achievable score before any modelling choice is made. We would rather report a modest number we can fully account for than an impressive one built on leakage.

11.2 Practical interpretation

What could someone actually do with this model, given its measured performance?

At this operating point the model is an indicator, not a decision tool. A passenger flying a major carrier on an evening departure could reasonably read a major-delay warning as a prompt to check again closer to departure or to leave slack in an onward connection, but not as a reason to rebook, because a major warning is correct roughly one time in nine (precision 0.1066) and 39.7% of on-time flights receive some warning.

Its most defensible use is aggregate rather than individual: the schedule-level patterns it confirms (the 2.7-fold rise in delay rate across the day, the carrier spread, the December peak) are stable, measured on 600,002 flights, and useful for planning conversations. The per-flight prediction is not reliable enough to carry consequences for a named passenger, and Section 9.4 shows that its reliability is not even distributed evenly across them.

11.3 Responsible use boundary

This model should NOT be used to	Reason
Determine compensation or refunds for individual passengers	Accuracy is far too low to support decisions with financial consequences for individuals
Rank, penalise or publicly compare airlines	Our findings are associational. Delay rates confound route mix, schedule timing and hub congestion
Provide guarantees to travellers	The model cannot support any promise about a specific flight
Operate in regions or periods unlike the training data	Distribution shift is demonstrated in Section 9.5

This model MAY reasonably be used to	Condition
Illustrate which scheduled factors associate with delay	Presented as association, never as cause
Offer a rough, clearly-labelled likelihood indication	Accompanied by an explicit statement of its accuracy

11.4 Future improvements

Improvement	Expected benefit	Difficulty
Add historical weather data by airport and date	Likely the largest single gain available. Weather is a dominant delay cause entirely absent from our features	Moderate: requires joining an external source
Add aircraft rotation history	Captures delay propagation between consecutive flights of the same airframe	High: requires tail-number tracking
Add airport congestion measures	Departures scheduled per hour at the origin airport	Low: derivable from the existing data
Simplify to two classes	Merging minor and major would likely raise recall on delays substantially, at the cost of severity information	Low
Probability outputs with a tuned threshold	Would let the false-alarm trade-off in Section 9.2 be set explicitly per stakeholder	Low

12. References and Contribution Statement

12.1 Dataset

US Bureau of Transportation Statistics, Office of Airline Information (2019). Reporting Carrier On-Time Performance, calendar year 2018. United States Department of Transportation. <https://www.transtats.bts.gov/>. Accessed via a consolidated per-year CSV re-upload hosted on Kaggle; the originating publisher is cited here and the re-upload recorded as the access route only.

12.2 Contribution statement

M1: ESRAT

ESRAT, ID 23-50916-1. Owned data provenance and auditing. Sourced the BTS 2018 on-time file, wrote `src/extract_subset.py`, which reduces the 851.6 MB, 7,213,446-row full-year file to the 1,796,121-row October-December subset by chunked reading rather than loading it whole, and wrote `src/data_loading.py`, which deletes the seventeen post-departure columns at load time and constructs the three-class target. Wrote `src/data_audit.py` and its seven checks, produced `data_dictionary.md`, and drafted report Sections 2 and 3. Identified the single impossible `CRS_ELAPSED_TIME` value of -99 minutes and argued for reporting rather than deleting it. Reviewed M2's split logic before integration. Able to explain the full workflow, including the modelling and evaluation stages owned by M3 and M4.

M2: AUVROW

AUVROW, ID 22-48920-3. Owned exploratory analysis, split design, preprocessing and leakage control. Wrote `src/eda.py` and Figures 1 to 4, establishing the hour-of-day gradient that later proved to be the model's most-used feature. Wrote `src/split.py`, which splits on whole calendar days rather than rows, and `src/preprocessing.py`, implementing imputation, scaling and top-N encoding explicitly rather than through `sklearn.Pipeline`, a choice defended in Section 5.4. Wrote `src/leakage_check.py` and its six checks, and rewrote check 5 after it flagged a 21.78% feature-collision rate: rather than raising the threshold until the data passed, reframed it to test date disjointness and to report the collision rate as an INFO measurement. Drafted report Sections 4 and 5. Able to explain the full workflow.

M3: NABIL

NABIL, ID 24-60205-3. Owned baselines, model selection and tuning. Wrote `src/models.py` and `src/train.py`. Defined both baselines and insisted they be scored with the same metrics as the real models. Selected the three candidates for their different failure modes rather than their expected scores, applied balanced class weighting to the two models that support it, and documented in advance, before the test set was opened, that KNN would under-predict the rare classes because `KNeighborsClassifier` offers no `class_weight` parameter. Ran all tuning by 3-fold stratified cross-validation inside the training data on `f1_macro`, retained

the losing configurations in results/metrics/tuning_log.csv, and checked every model for overfitting (largest train-to-CV gap 0.0270, Random Forest). Drafted Sections 6 and 7. Able to explain the full workflow.

M4: AREFIN

AREFIN, ID 22-48766-3. Owned evaluation, figures, error analysis and reproducibility. Wrote config.py, run_all.py, verify_reproducibility.py, src/evaluate.py, src/plots.py and src/error_analysis.py, plus README.md and requirements.txt. Opened the test set exactly once, after model selection was frozen, and withheld all test figures from M3 until then. Produced every result figure under stated fairness rules (colourblind-safe palette, counts and percentages in every confusion cell, axes starting at zero), and included Figure 7, which shows the do-nothing baseline beating the selected model on accuracy. Broke the 55,547 errors down by severity, hour and carrier, and raised the 6.2-fold error disparity between EV and DL passengers as a fairness problem in Section 9.4. Recorded that the by-month breakdown is empty because the test window sits inside a single month. Drafted Sections 8 to 11. Able to explain the full workflow.

Python Architecture:

1. External components

Four external libraries, declared in requirements.txt.

Library	Role in this project
pandas	Every table. Reads the CSV, builds the target column, filters rows, groups by day, produces the summary/metrics .csv files.
numpy	Vectorised numeric work, np.select for the 3-class labels, array maths in error analysis, random generation for the fake sample data.
scikit-learn	The models (DummyClassifier, LogisticRegression, KNeighborsClassifier, RandomForestClassifier), tuning (GridSearchCV, StratifiedKFold), and metrics.
matplotlib	All .png output. Used with the non-interactive Agg backend (set at the top of eda.py, plots.py, error_analysis.py) so figures write to disk without opening a window.

The preprocessing is written by hand in pandas.

2. How data flows

Each stage feeds the next; the pipeline runs top to bottom.

Stage	What happens
CSV → pandas DataFrame	Raw file loaded into memory.
↓ drop_forbidden_columns()	remove leakage cols (ARR_DELAY components etc.)

Stage	What happens
↓ <code>create_target()</code>	<code>np.select</code> on <code>ARR_DELAY</code> → 0 / 1 / 2
↓ <code>engineer_features()</code>	<code>.dt</code> accessor → <code>MONTH</code> , <code>DAY_OF_WEEK</code> ; <code>//100</code> → <code>DEP_HOUR</code>
↓ <code>apply_row_cap()</code>	<code>groupby(day).sample(frac=...)</code> with seed 42
↓ <code>time_aware_split()</code>	cut by <code>DATE</code> , not by row
↓ <code>prepare_features()</code>	learn on train → apply to train AND test
↓ <code>sklearn .fit()</code> / <code>.predict()</code>	numpy arrays / DataFrames into the estimators
↓ <code>metrics</code> → <code>pandas</code>	written to <code>results/metrics/*.csv</code>
↓ <code>matplotlib</code>	written to <code>results/figures/*.png</code>